RepoReaper - Water Curse

Digital Forensics & Incident Analysis Report — Supply Chain Compromise Analysis

Incident Target	Developer Workstation (Host: `DESKTOP-POKO2F4`, User: `DemonSlayer`)
Infection Vector	Weaponized Open-Source Repository (GitHub Supply Chain Attack)
Primary Artifacts Analyzed	Browser History, NTFS $MFT / $LogFile, Prefetch (.pf), Windows Event Logs, Registry Hives
Key Tools Employed	FTK Imager, DB Browser for SQLite, DCode, Timeline Explorer, Registry Explorer, Event Log Explorer
Investigative Outcome	Full attack chain reconstructed: delivery via ZIP, execution via PowerShell & 7-Zip, privilege escalation via UAC bypass, persistence via scheduled tasks, defense evasion via System Restore disabling & process hollowing, and C2 exfiltration via Telegram API.

1. Executive Summary

A sophisticated supply chain attack targeted internal software developers through a weaponized open-source repository masquerading as a legitimate script-to-scene conversion tool on GitHub. Upon compilation, the repository dropped a multi-stage malware package. Within minutes, the payload established persistence via scheduled tasks, escalated privileges through a UAC bypass leveraging auto-elevating system binaries, and executed encrypted in-memory payloads using process hollowing. Additionally, the malware altered system registry settings to disable recovery features, executed anti-analysis checks against security software, and exfiltrated sensitive data to remote infrastructure over legitimate platform APIs.

2. Scope and Evidence

The scope of this investigation encompasses forensic analysis of host artifacts recovered from the infected developer workstation. Evidence items analyzed include web browser SQLite databases, NTFS Master File Table ($MFT) exports, prefetch execution logs, system and software registry hives, and Windows Event Logs (including PowerShell operational and Windows Filtering Platform network logs).

3. Background: Weaponized Repository Supply Chain Attack

Supply chain attacks targeting development environments typically deliver malicious code embedded in build scripts or dependencies. In this scenario, the attacker hosted a compromised repository containing open-source code. When compiled, embedded installation routines dropped secondary payloads, leveraged living-off-the-land binaries (LOLBins) for privilege escalation, and deployed obfuscated components contained within custom archives.

4. Investigation Phases

Phase 1 — Initial Access and Web Artifact Recovery

Forensic examination began with analyzing browser artifacts to determine the initial infection vector. Microsoft Edge browser history stored within the `History` SQLite database was analyzed using DB Browser for SQLite.

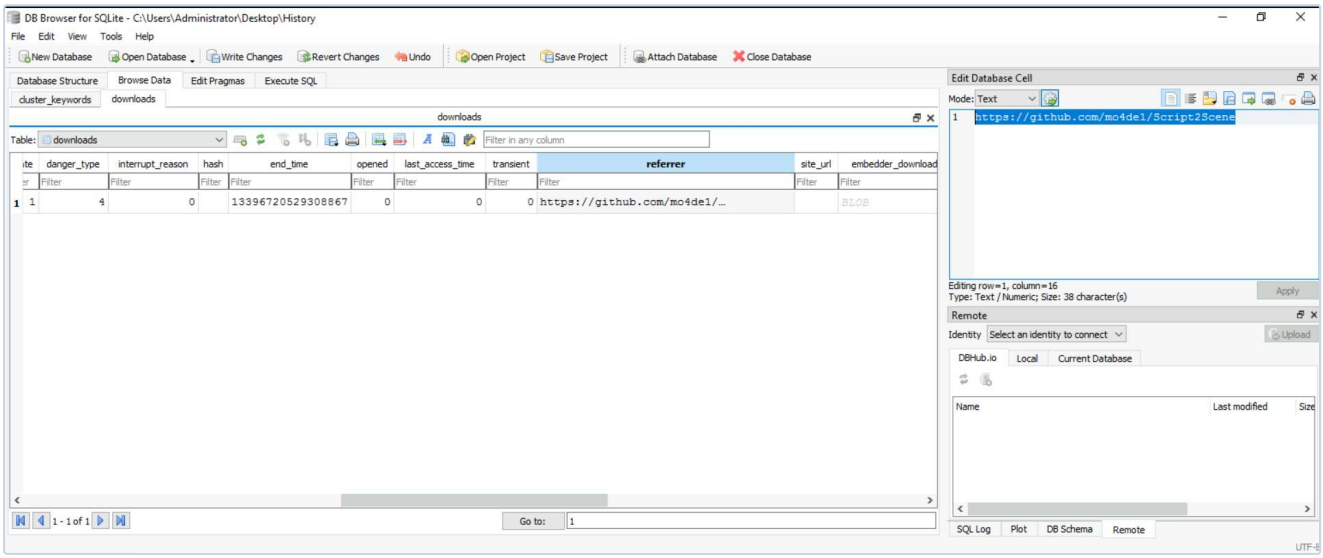


Figure 1. DB Browser for SQLite displaying the download referrer URL in the Edge History database.

The download records identified the source repository URL as `https://github.com/mo4de1/Script2Scene`, confirming that the compromised open-source repository served as the initial access point.

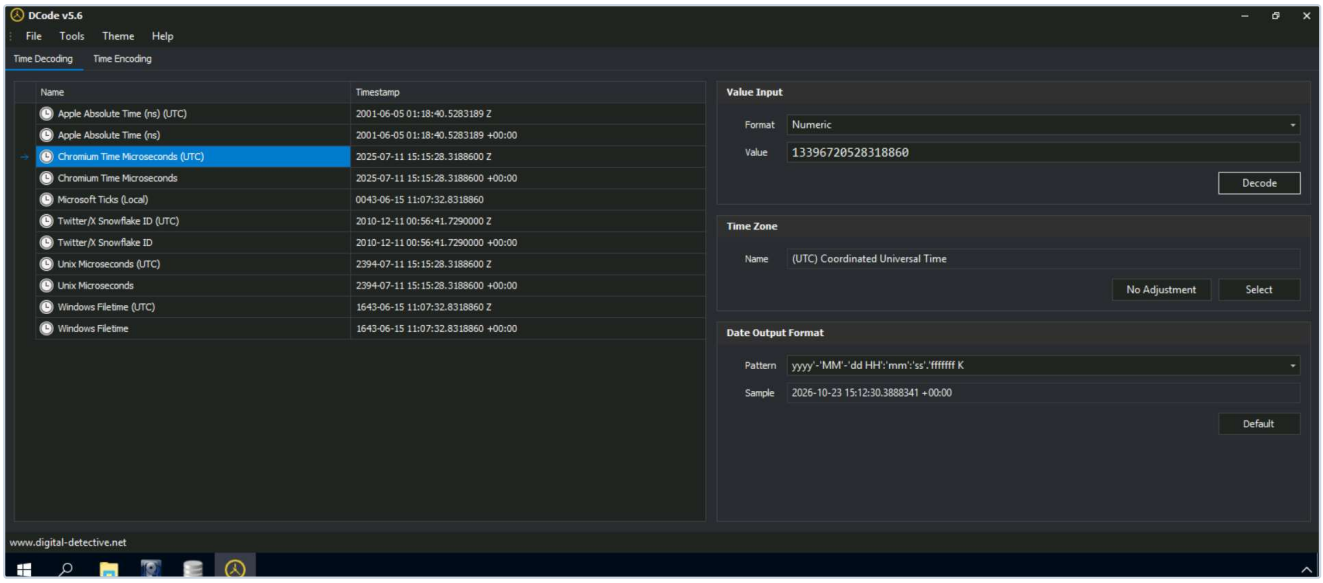


Figure 2. Converting the Chromium microsecond download timestamp in DCode.

The download timestamp `13396720526318161` was extracted from the `start_time` column and decoded using DCode, establishing the initial download time as **2025-07-11 15:15:28 UTC**.

Phase 2 — Initial Execution & Payload Dropping

Following file download and project execution, NTFS metadata and event logs were cross-referenced to track initial payload execution. Analysis of `Windows PowerShell.evtx` using Event Log Explorer revealed the execution of the first-stage PowerShell script.

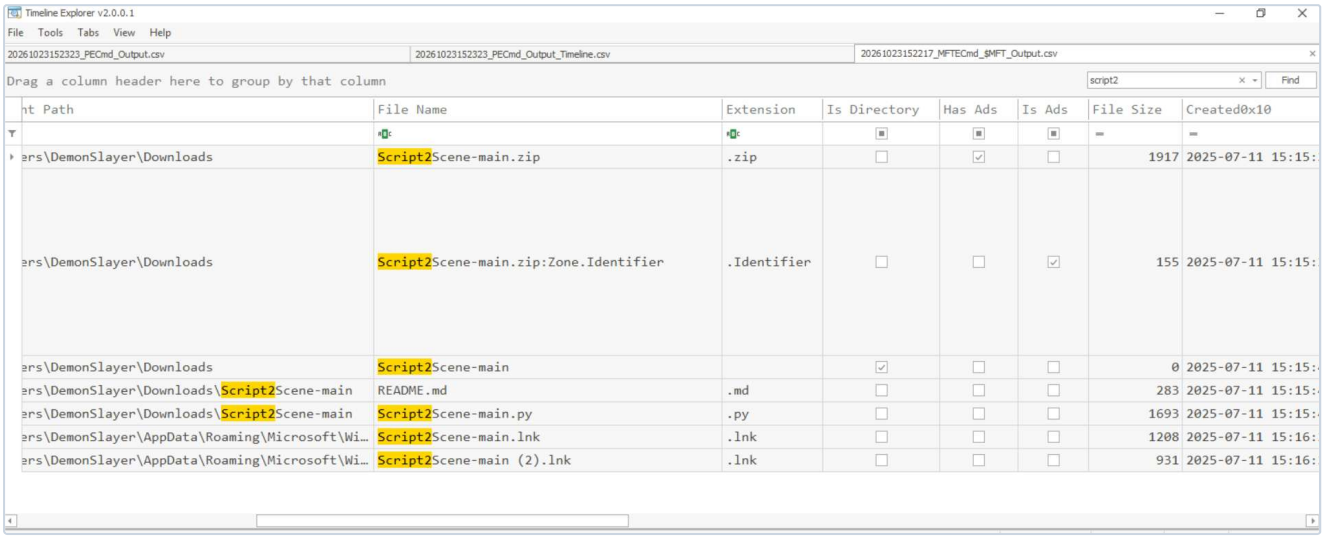


Figure 3. Timeline Explorer showing MFT creation events for Script2Scene files and shortcuts.

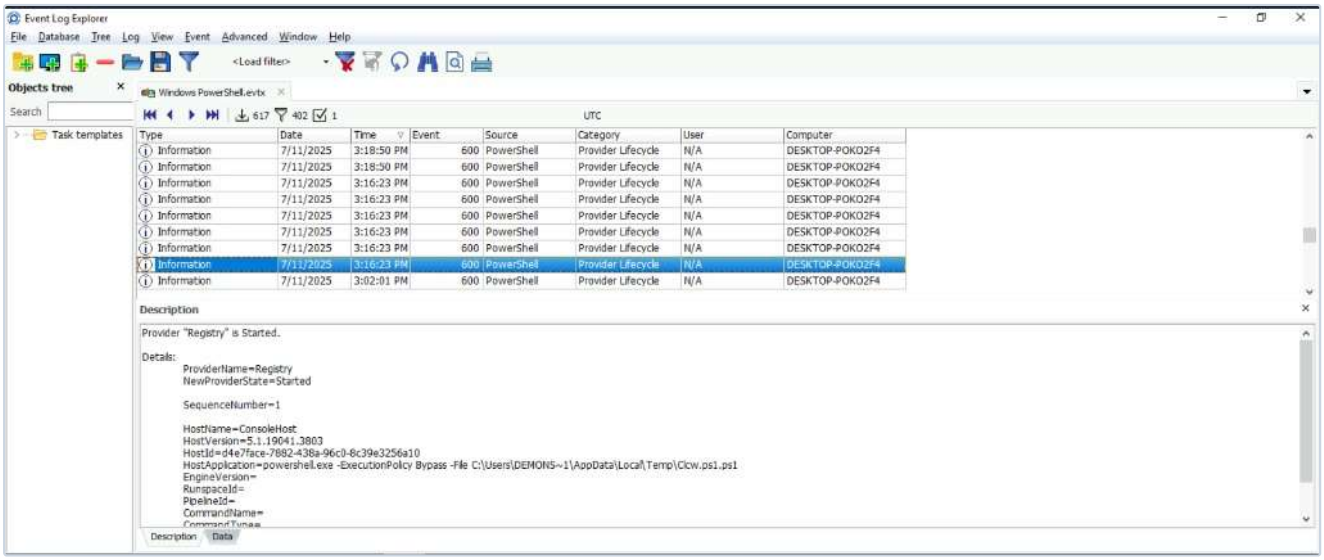


Figure 4. Event ID 600 logging execution of Clcw.ps1.ps1 with ExecutionPolicy Bypass.

The script `Clcw.ps1.ps1` was launched from the `AppData\Local\Temp` directory with execution policy bypass parameters shortly after project compilation.

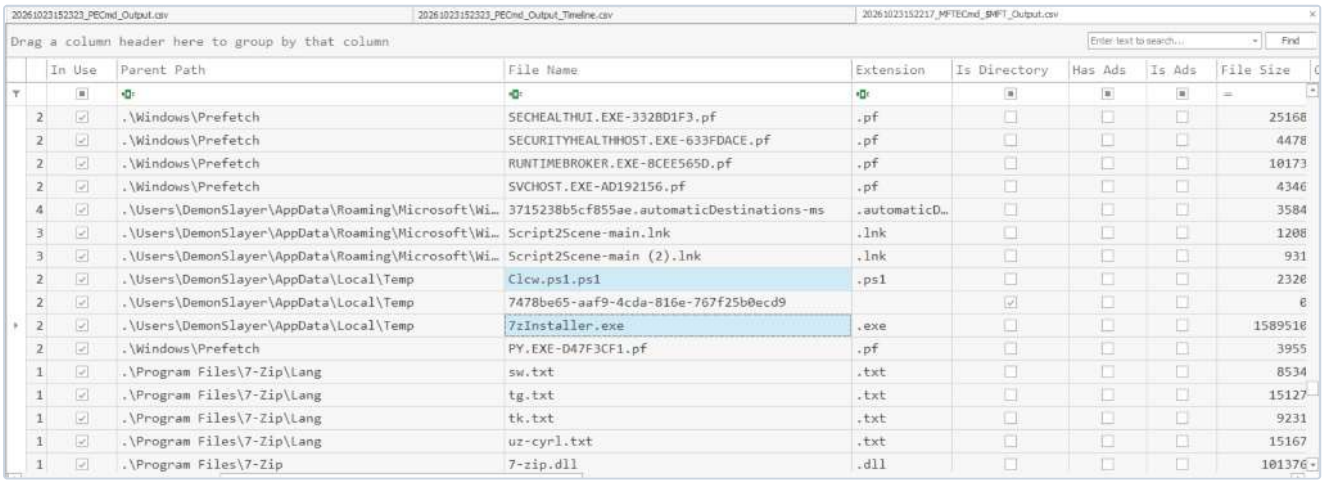


Figure 5. Master File Table timeline showing 7zInstaller.exe and the encrypted archive.

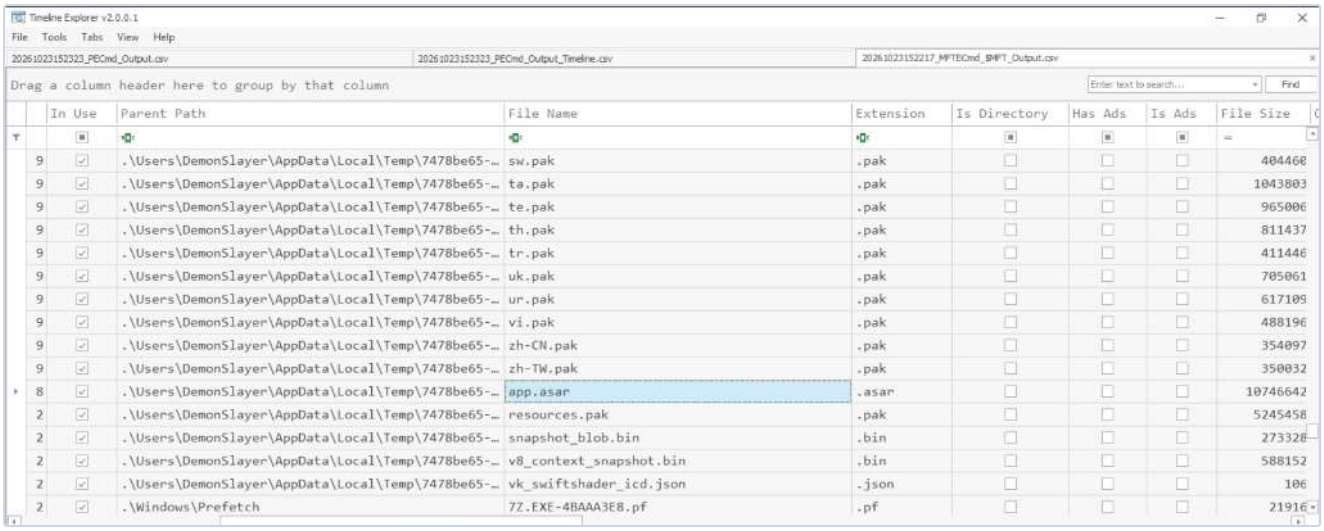


Figure 6. MFT records detailing the extraction of Electron application assets into Temp subdirectories.

Analysis of $MFT records confirmed that `Clcw.ps1.ps1` dropped `7zInstaller.exe` and a password-protected 7-Zip archive named `4359ce22-2f6d-43b8-b34d-063d74715562.7z` at 15:16:35 UTC. The archive was subsequently extracted into `AppData\Local\Temp\7478be65-aaf9-4cda-816e-767f25b0ecd9`, laying the groundwork for the Electron-based application framework.

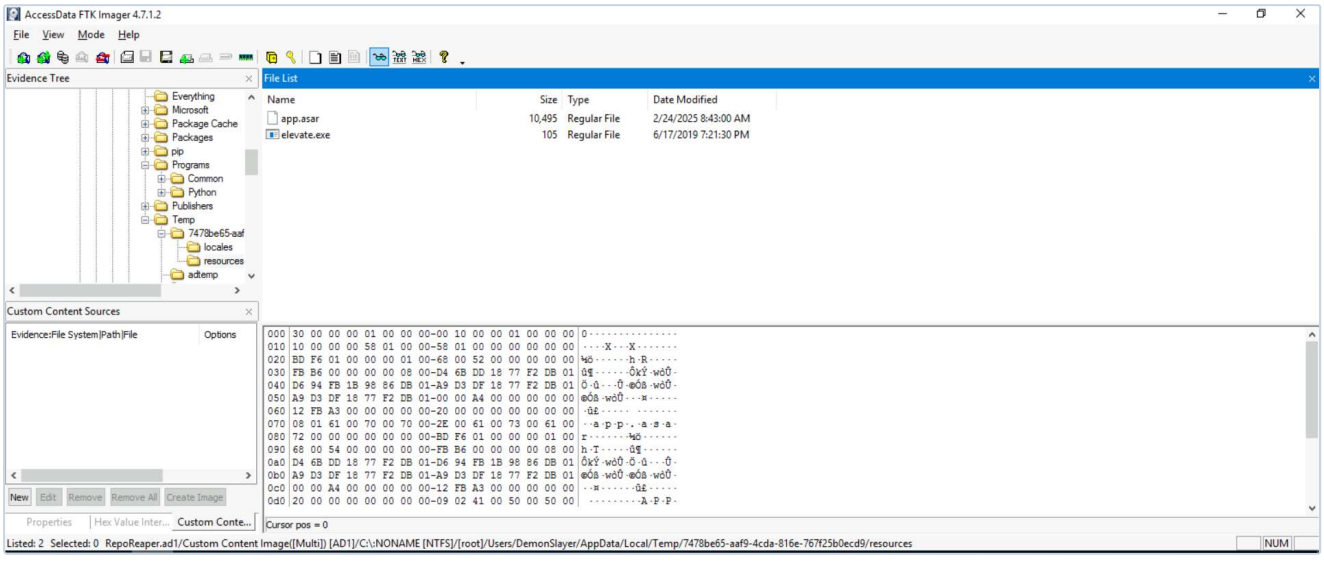


Figure 7. FTK Imager displaying the extracted app.asar file inside the resources directory.

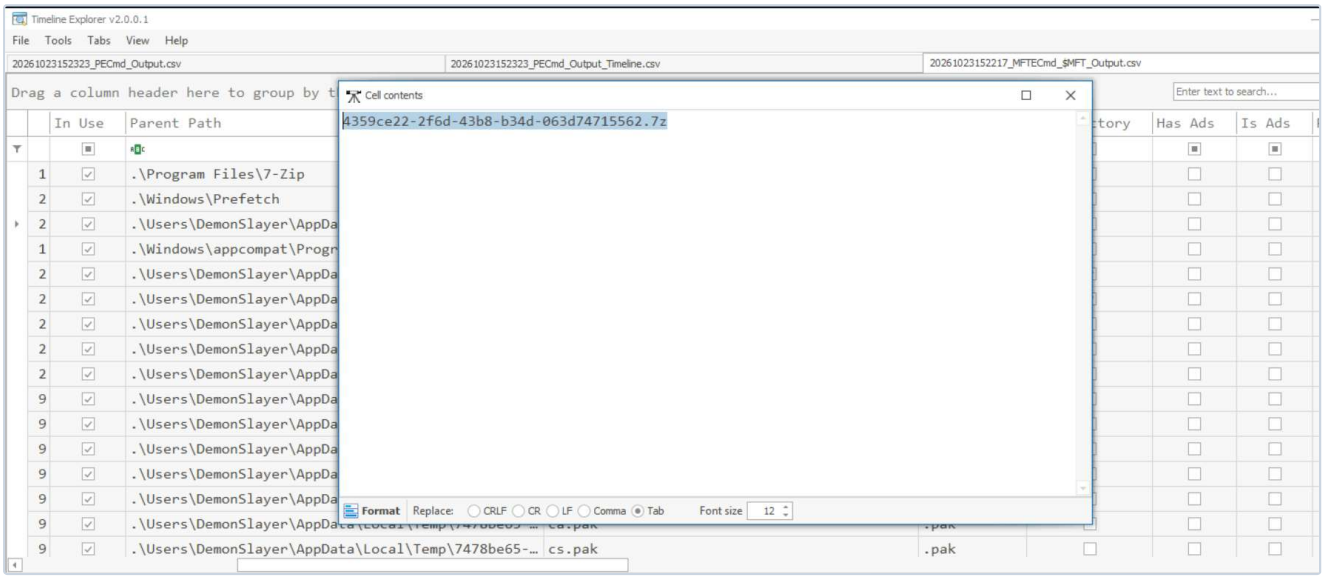


Figure 8. Timeline Explorer detail confirming the exact 7z archive container name.

Phase 3 — Core Payload Analysis & Hashing

The extracted framework contained `app.asar`, an Atom Shell Archive storing the Electron application's core JavaScript files. Using the Asar utility, `app.asar` was unpacked to inspect its contents.

```
v3.2.0
PS C:\Users\Administrator\Desktop\Start Here\Tools\Miscellaneous> asar.cmd extract .\app.asar extracted_app
PS C:\Users\Administrator\Desktop\Start Here\Tools\Miscellaneous> dir


    Directory: C:\Users\Administrator\Desktop\Start Here\Tools\Miscellaneous


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         8/27/2024   4:42 PM                DidierStevensSuite
d-----         8/26/2024   1:11 PM                dnSpy
d-----        10/23/2026   4:05 PM                extracted_app
d-----         8/26/2024   1:11 PM                ILSpy
d-----         8/23/2024   6:57 PM                impacket
d-----         8/26/2024   1:13 PM                KAPE
d-----          8/8/2024   2:36 PM                Radare2
d-----        10/15/2003   8:34 AM                steghide
d-----         8/27/2024   1:59 PM                SysinternalsSuite
-a----         8/14/2024   5:48 PM            872 010 Editor.lnk
-a----         8/15/2024   1:40 PM           1423 4n6 Outlook Forensics Wizard.lnk
-a----         8/21/2024   2:45 PM           1636 apimonitor-x64.exe.lnk
-a----         2/24/2025   8:43 AM       10746642 app.asar
-a----        10/23/2026   4:02 PM           2267 asar.lnk
-a----         8/14/2024   5:35 PM           1413 DB Browser (SQLCipher).lnk
-a----         8/14/2024   5:35 PM           1398 DB Browser (SQLite).lnk
-a----         8/14/2024   9:21 PM           1235 DCode v5.6.lnk
-a----         8/21/2024   2:54 PM            924 HxD.lnk
-a----         8/15/2024   1:30 PM            956 IDA Freeware 8.4.lnk
-a----         8/15/2024  12:27 PM      114693170 jadx.exe
-a----         8/15/2024   5:08 PM           1224 Outlook Recovery Kit.lnk
-a----         8/14/2024   9:34 PM           1302 PST Walker.lnk
-a----         8/15/2024   1:22 PM           1061 R-Studio.lnk
-a----         8/15/2024   3:16 PM           1123 Resource Hacker.lnk
-a----         8/14/2024  10:19 PM           1434 Visual Studio Code.lnk
```

Figure 9. Unpacking app.asar via PowerShell CLI into the extracted_app directory.

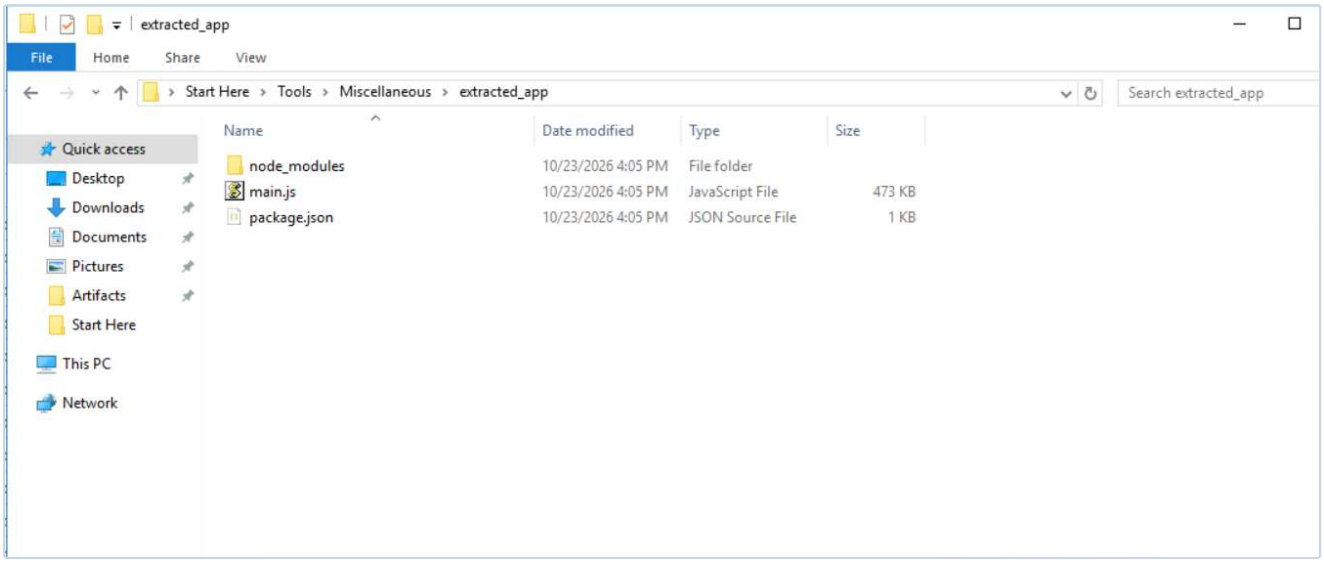


Figure 10. Extracted application folder containing main.js and package.json.

After extraction, the MD5 hash of `main.js` was computed using PowerShell's `Get-FileHash` cmdlet.

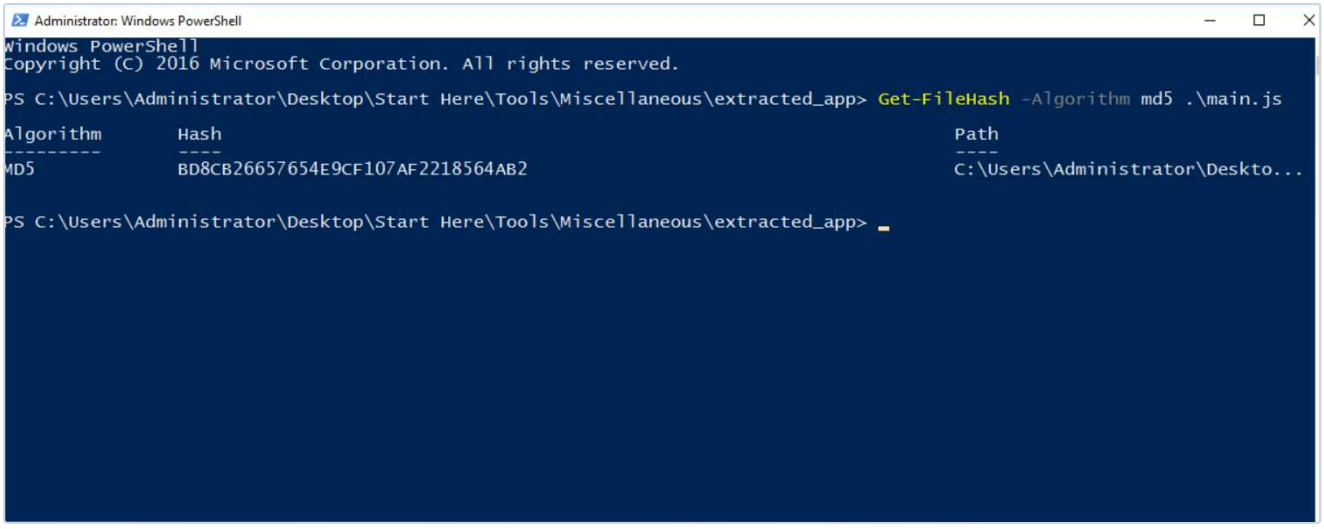


Figure 11. Computing MD5 cryptographic hash for main.js.

The resulting hash, `BD8CB26657654E9CF107AF2218564AB2`, uniquely identifies the malicious JavaScript wrapper responsible for orchestrating subsequent execution stages.

Phase 4 — Persistence Mechanisms

To establish persistence across system reboots, the malware modified system scheduled tasks within the `SOFTWARE` registry hive under `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\Tasks`.

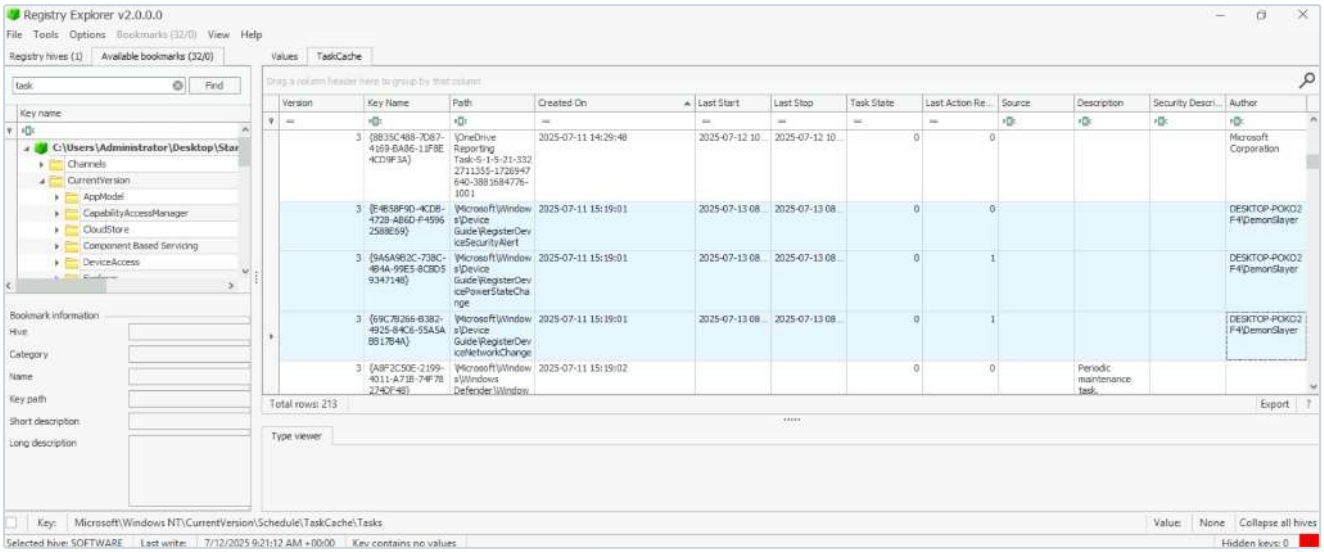


Figure 12. Registry Explorer viewing scheduled task entries in the TaskCache key.



		1001									
3	{E4B58F9D-4CDB-4728-AB6D-F4596 2588E69}	\Microsoft\Windows\Device Guide\RegisterDeviceSecurityAlert	2025-07-11 15:19:01	2025-07-13 08...	2025-07-13 08...	0	0				DESKTOP-POKO2F4\DemonSlayer
3	{9A6A9B2C-738C-4B4A-99E5-8CBD5 9347148}	\Microsoft\Windows\Device Guide\RegisterDevicePowerStateChange	2025-07-11 15:19:01	2025-07-13 08...	2025-07-13 08...	0	1				DESKTOP-POKO2F4\DemonSlayer
3	{69C7B266-B382-4925-84C6-55A5A BB17B4A}	\Microsoft\Windows\Device Guide\RegisterDeviceNetworkChange	2025-07-11 15:19:01	2025-07-13 08...	2025-07-13 08...	0	1				DESKTOP-POKO2F4\DemonSlayer

Figure 13. Time-filtered registry analysis highlighting tasks created within the attack window.

Three scheduled tasks were identified: `Register DeviceSecurityAlert`, `Register DevicePowerStateChange`, and `Register DeviceNetworkChange`. These names were chosen to masquerade as legitimate Windows maintenance routines.

Phase 5 — Privilege Escalation via UAC Bypass

The attack chain leveraged User Account Control (UAC) bypass mechanisms to elevate privileges without triggering user prompts. Analysis of `Microsoft-Windows-PowerShell/Operational` event logs (Event ID 4104) identified protocol handler hijacking.

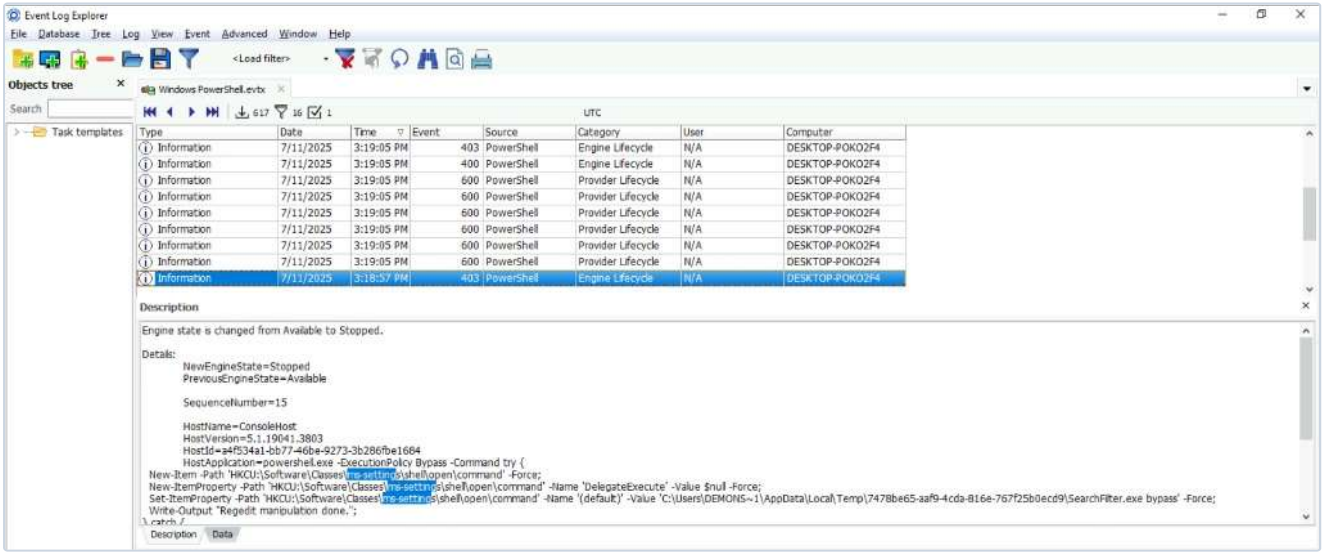


Figure 14. Event log record showing modification of the ms-settings registry command key.

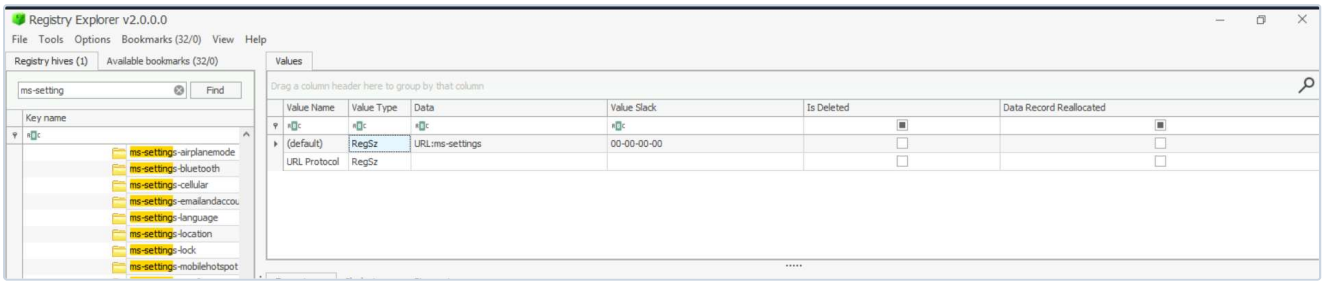


Figure 15. Registry verification of hijacked ms-settings protocol handler.

The malware hijacked the `HKCU\Software\Classes\ms-settings\shell\open\command` key to execute `SearchFilter.exe`. Immediately following this modification, `ComputerDefaults.exe` was invoked.

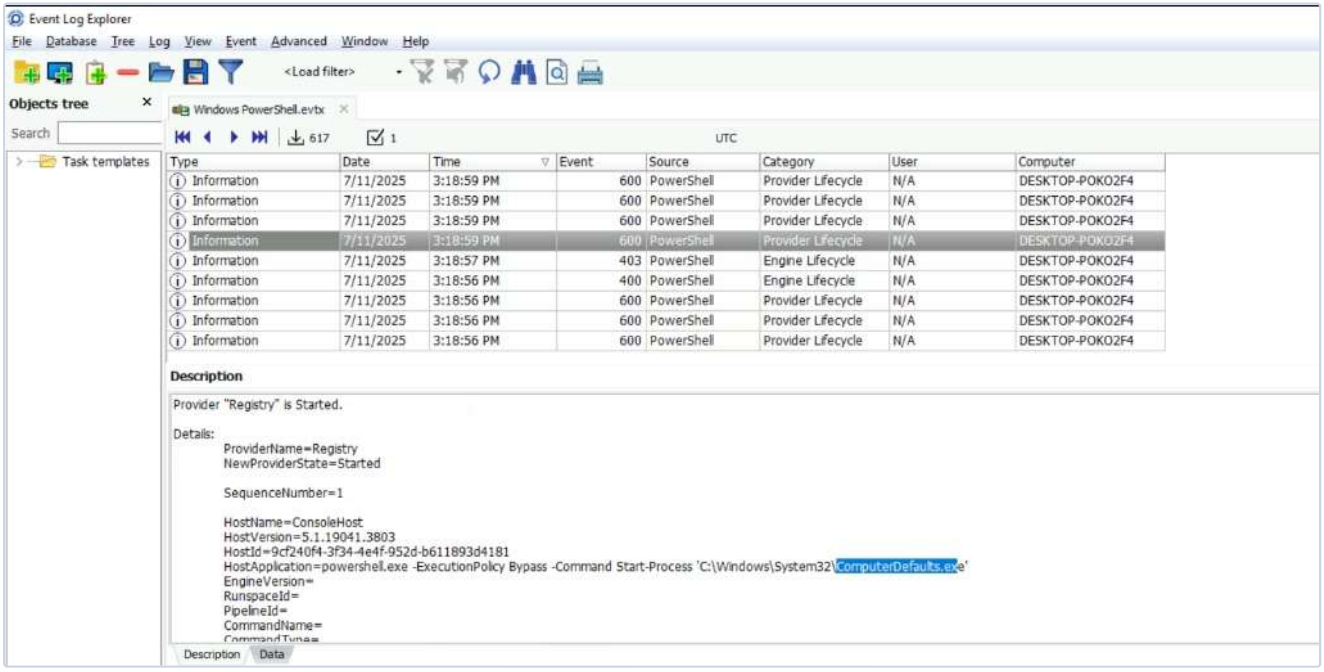


Figure 16. Execution of auto-elevating binary ComputerDefaults.exe to trigger UAC bypass.

Because `ComputerDefaults.exe` is an auto-elevating system binary that trusts the `ms-settings` protocol, launching it automatically executed the attacker's payload with high integrity privileges.

Phase 6 — Defense Evasion & Anti-Analysis

To impede recovery and evasion detection, the malware modified system restore settings and performed environment checks.

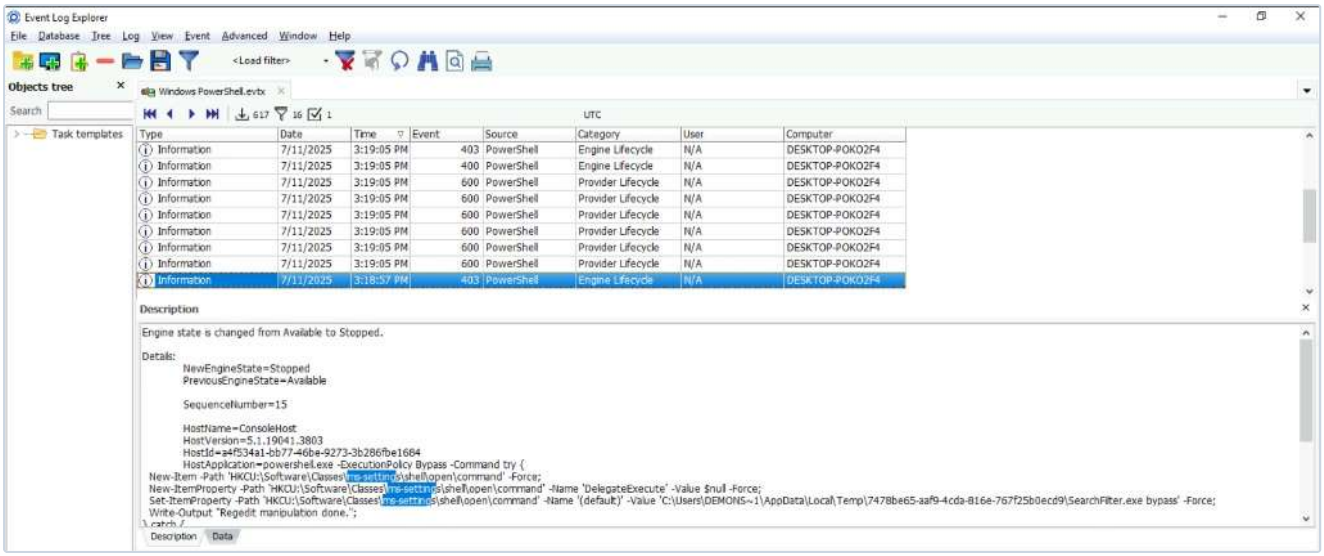


Figure 17. Registry Explorer showing DisableSR and DisableConfig values set to 1.

At **2025-07-11 15:19:04 UTC**, the malware set `DisableSR` and `DisableConfig` to `1` under `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\SystemRestore`, disabling Windows System Restore.

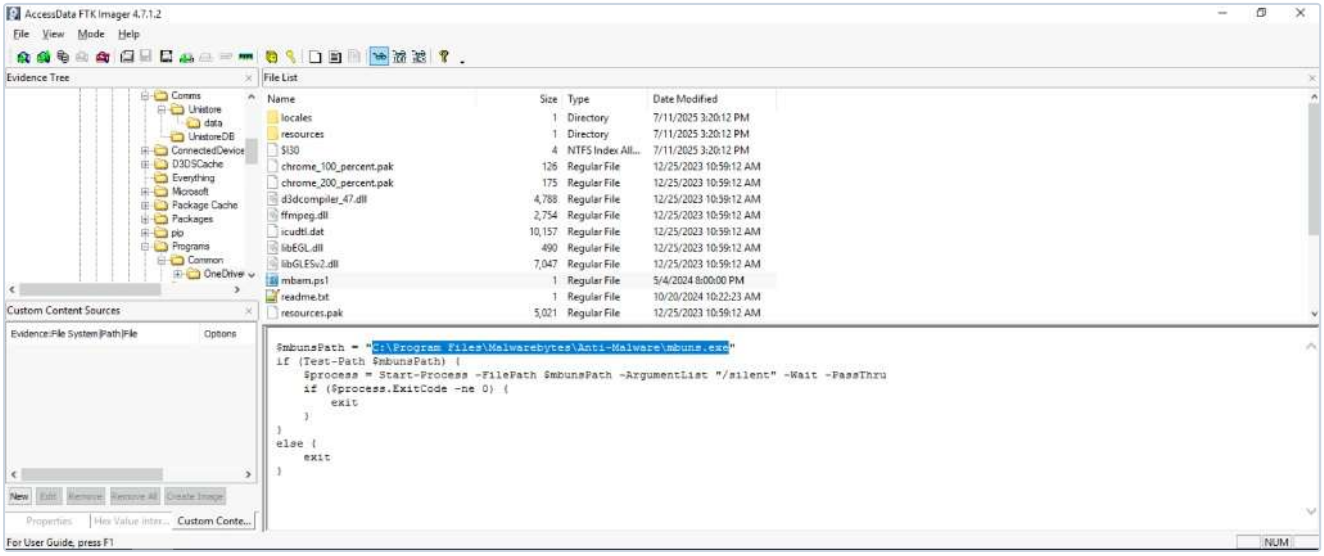


Figure 18. FTK Imager displaying the anti-analysis check in mbam.ps1.

Further inspection of scheduled task actions revealed execution of `mbam.ps1`, which performed a `Test-Path` check for `C:\Program Files\Malwarebytes Anti-Malware\mbuns.exe` to detect security software prior to full execution.

Phase 7 — Encrypted Payload Loading & Process Hollowing

The malware deployed three AES-encrypted payload components stored in the `OneDriveCloud\resources\manifest` directory: `boot_f`, `kernel_f`, and `thread_f`.

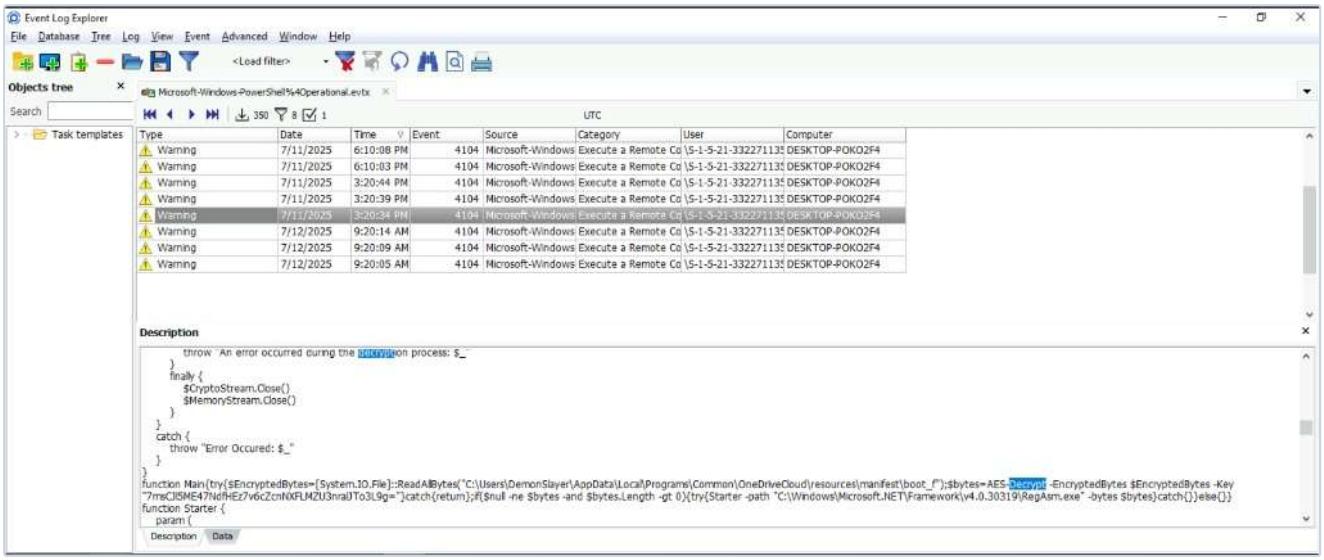


Figure 19. PowerShell Event ID 4104 capturing AES decryption routines for boot_f.

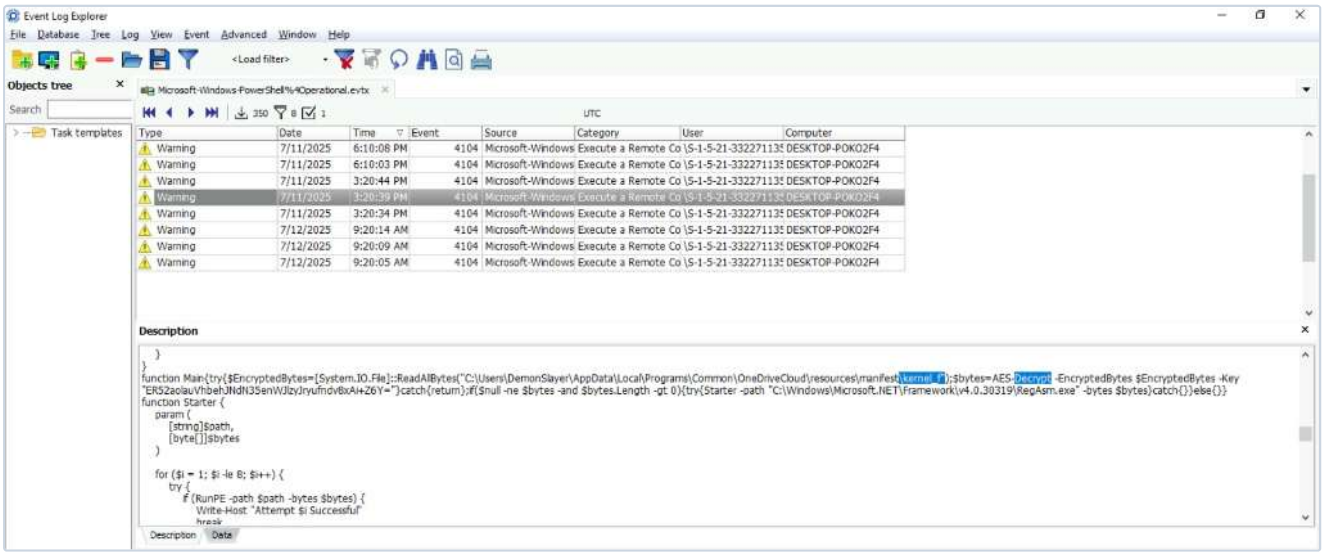


Figure 20. Event log detail showing decryption and injection loop for kernel_f.

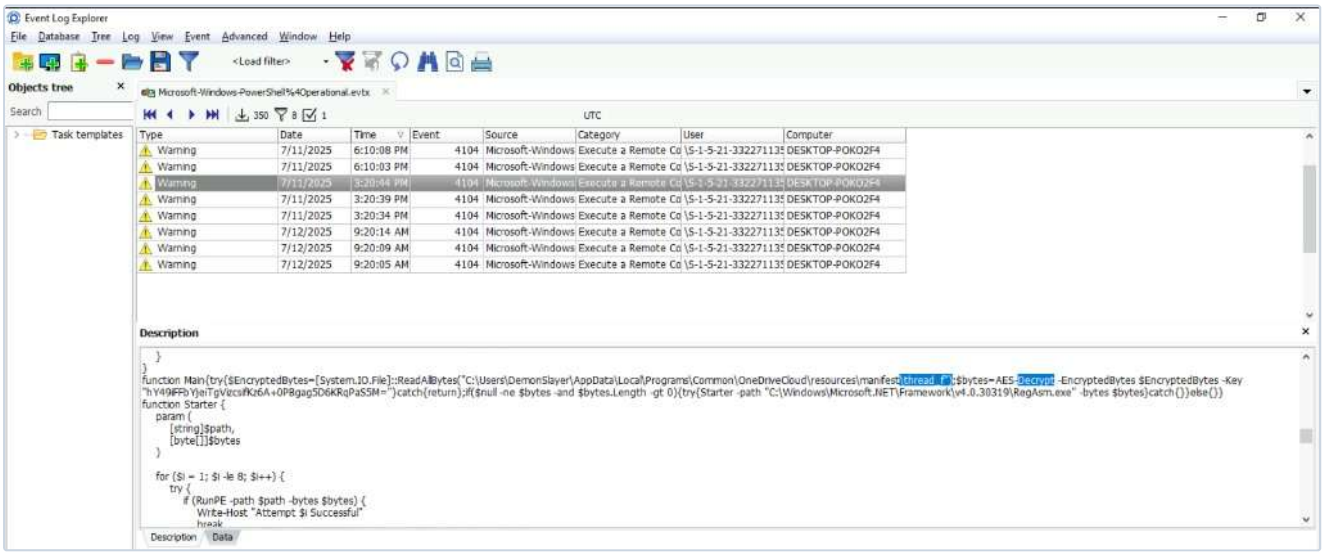


Figure 21. Event log showing thread_f decryption routine using hardcoded key and salt.

The scripts decrypted the payloads in memory and utilized process hollowing (RunPE technique) to inject code into suspended instances of `RegAsm.exe` (located in `Framework\v4.0.30319`).

Phase 8 — Binary Masquerading

Analysis of execution artifacts via PECmd and $MFT parsing revealed secondary binary masquerading.

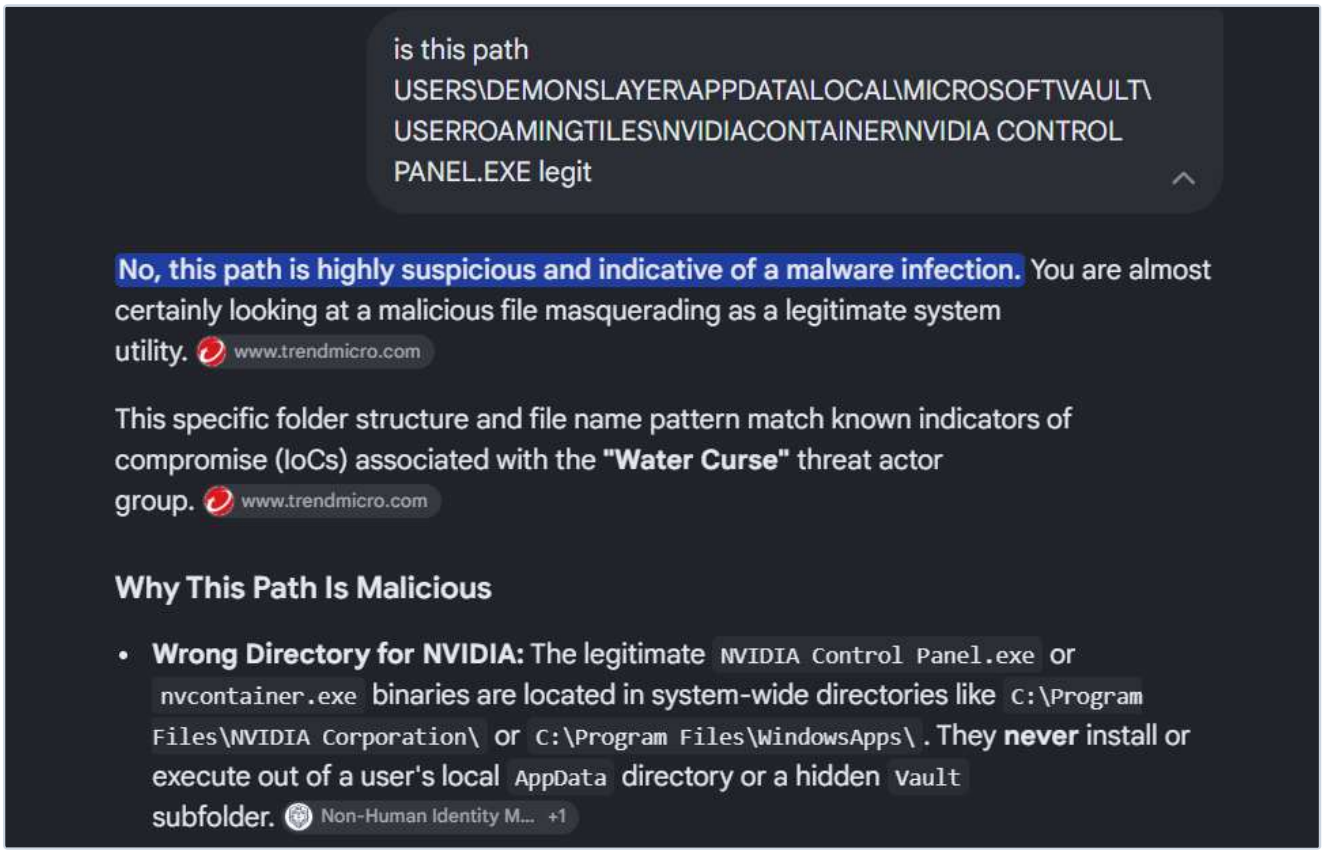


Figure 22. Verification of non-standard executable path mimicking legitimate software.

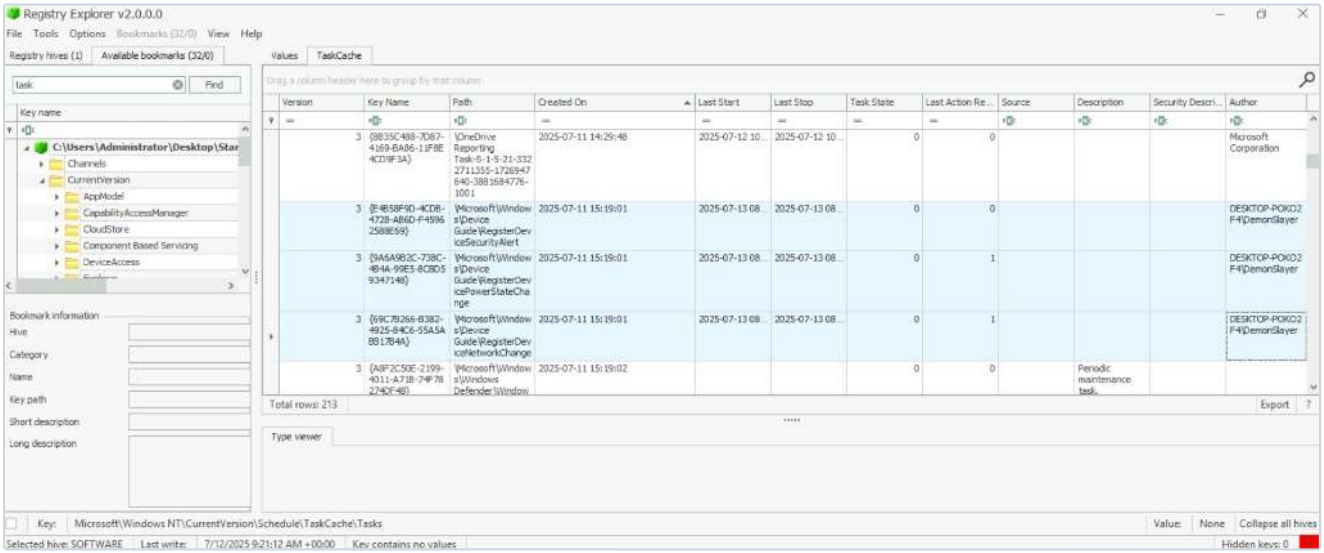


Figure 23. Timeline Explorer prefetch entries for NVIDIA CONTROL PANEL.EXE.

An executable named `NVIDIA CONTROL PANEL.EXE` was dropped in `AppData\Local\Microsoft\Vault\UserRoamingTiles\NVIDIAContainer`. This location is non-standard for legitimate NVIDIA software, proving binary masquerading.

Phase 9 — Command & Control (C2) Infrastructure

Network connections were established by both the masqueraded binary and the hollowed `RegAsm.exe` process. Outbound network traffic was analyzed via Windows Filtering Platform Event ID 5156 logs in `Security.evtx`.

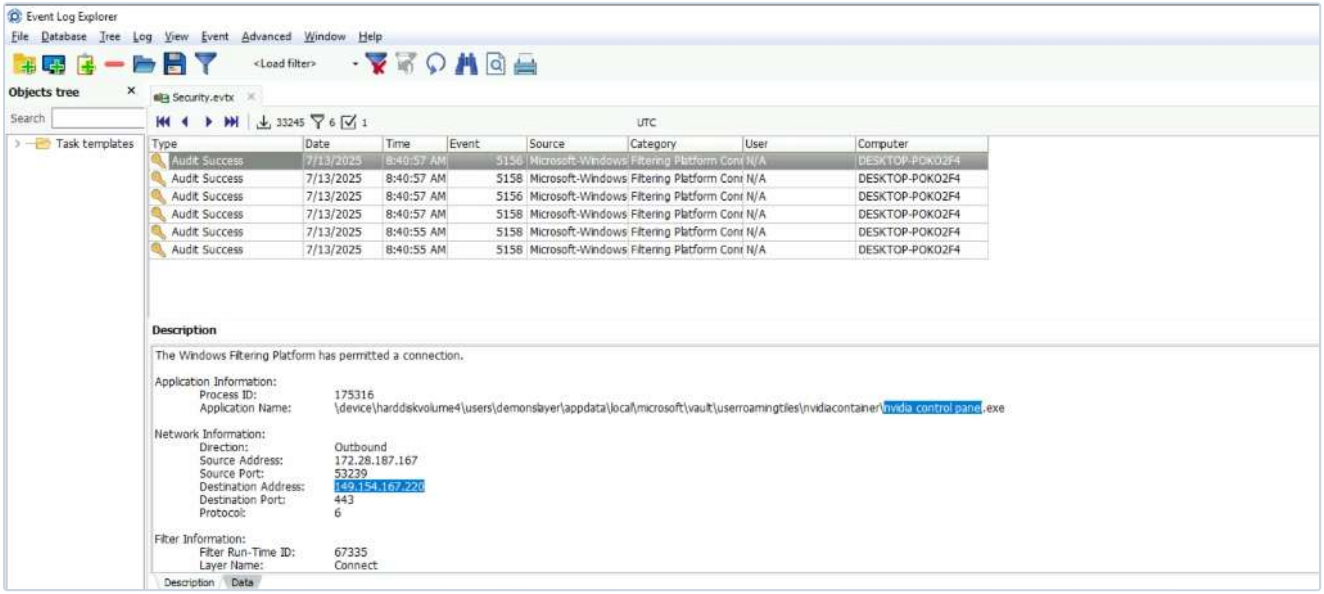


Figure 24. Event ID 5156 logging outbound HTTPS connection from NVIDIA CONTROL PANEL.EXE to IP 149.154.167.220.

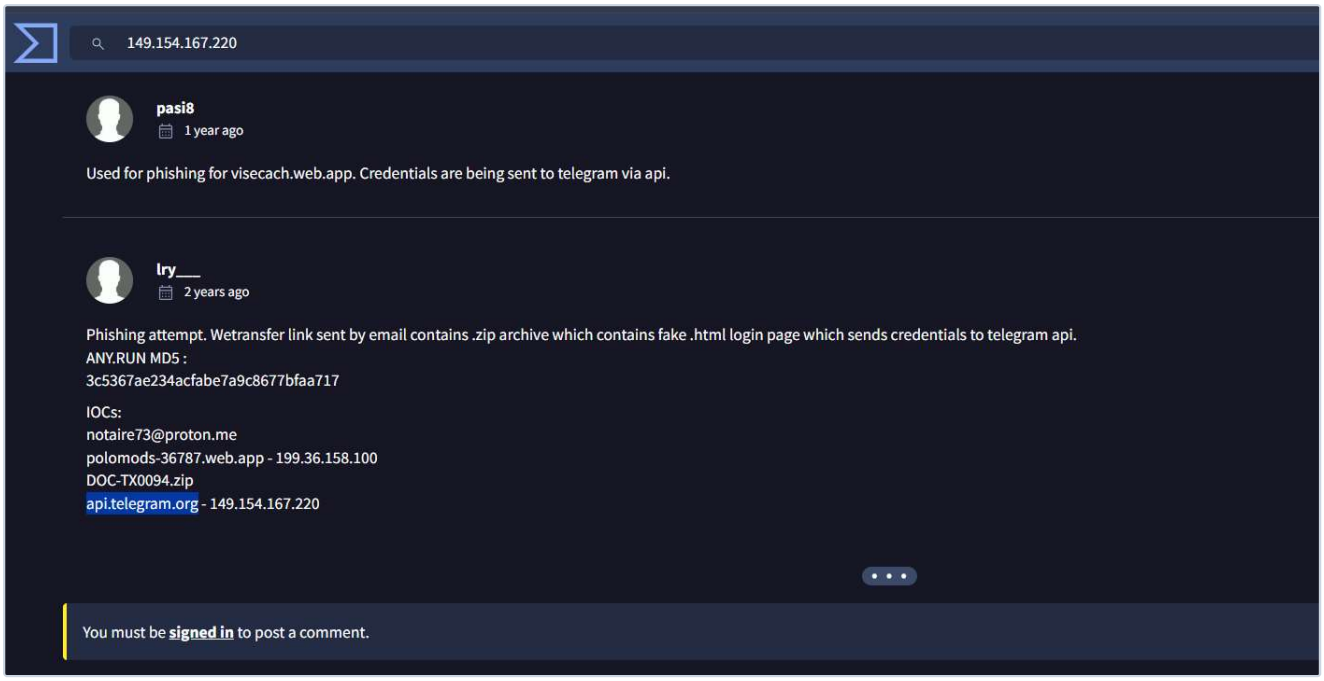


Figure 25. OSINT correlation linking IP 149.154.167.220 to api.telegram.org.

The process `NVIDIA CONTROL PANEL.EXE` established connections to `149.154.167.220` on TCP port 443. Threat intelligence lookup resolved this IP address to `api.telegram.org`, indicating data exfiltration over legitimate Telegram API infrastructure.

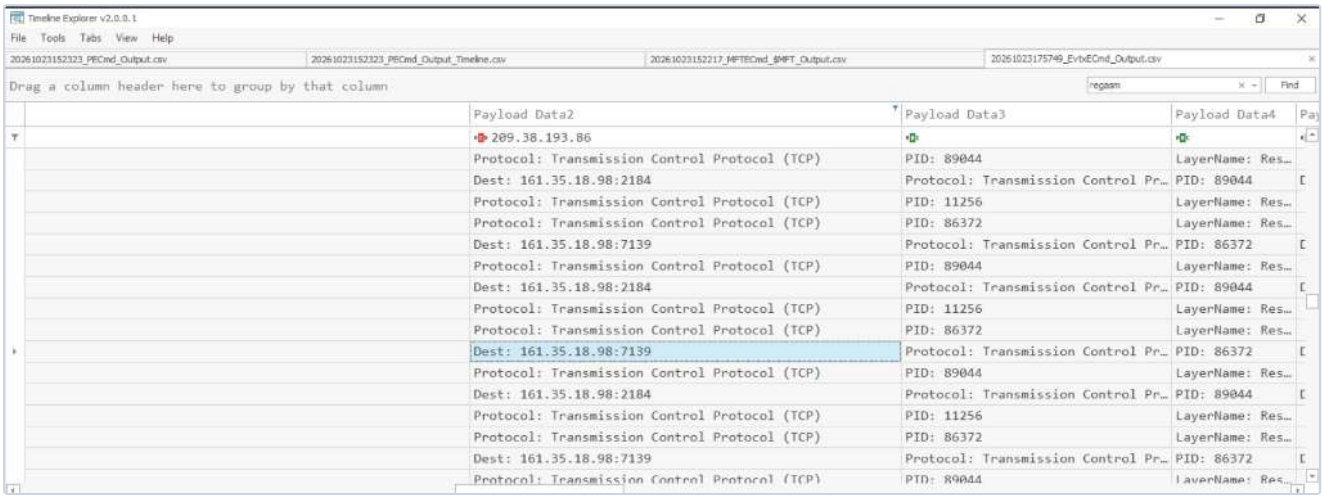


Figure 26. Event ID 5156 logging outbound connections from RegAsm.exe to external IPs.

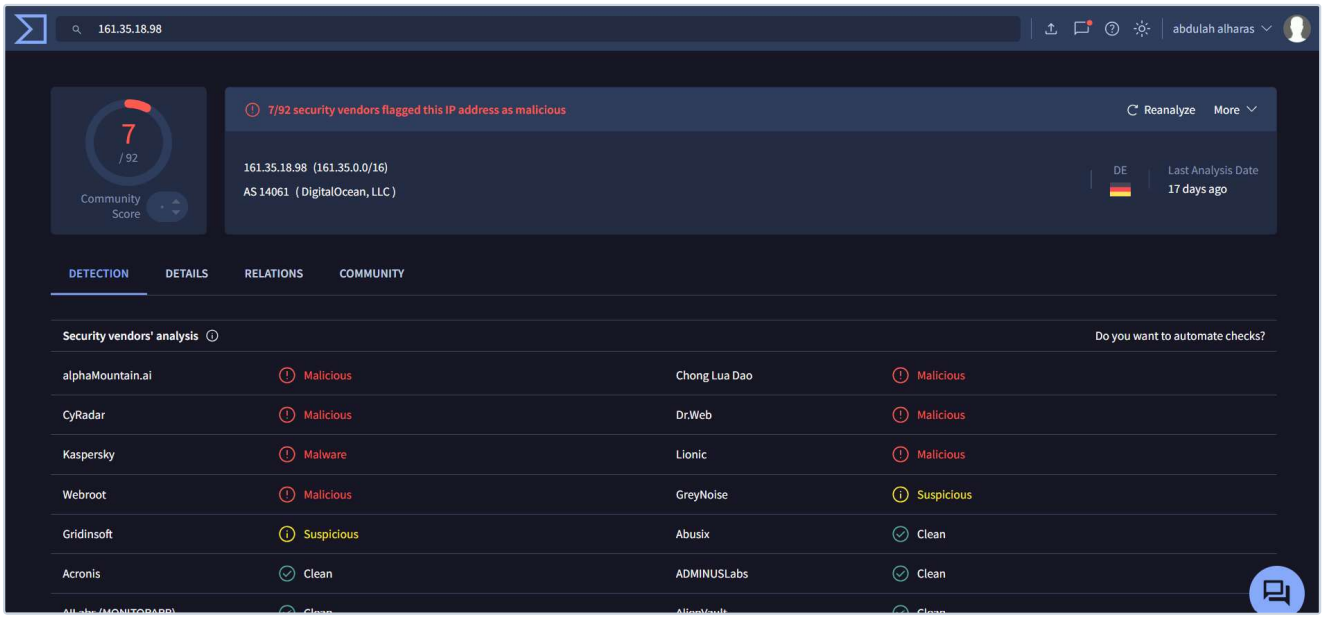


Figure 27. Threat intelligence vendor analysis for malicious C2 IP 161.35.18.98.

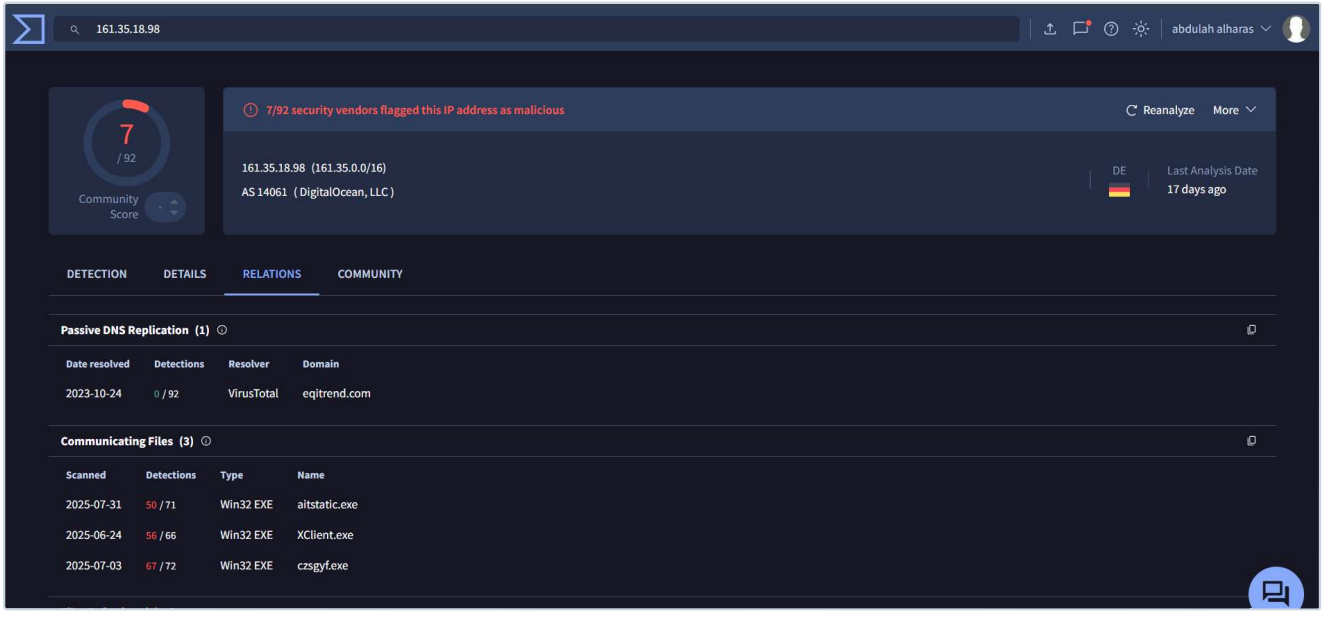


Figure 28. VirusTotal relations tab showing passive DNS resolutions and communicating files.



Figure 29. VirusTotal Graph Summary displaying IP resolutions and associated malware samples.

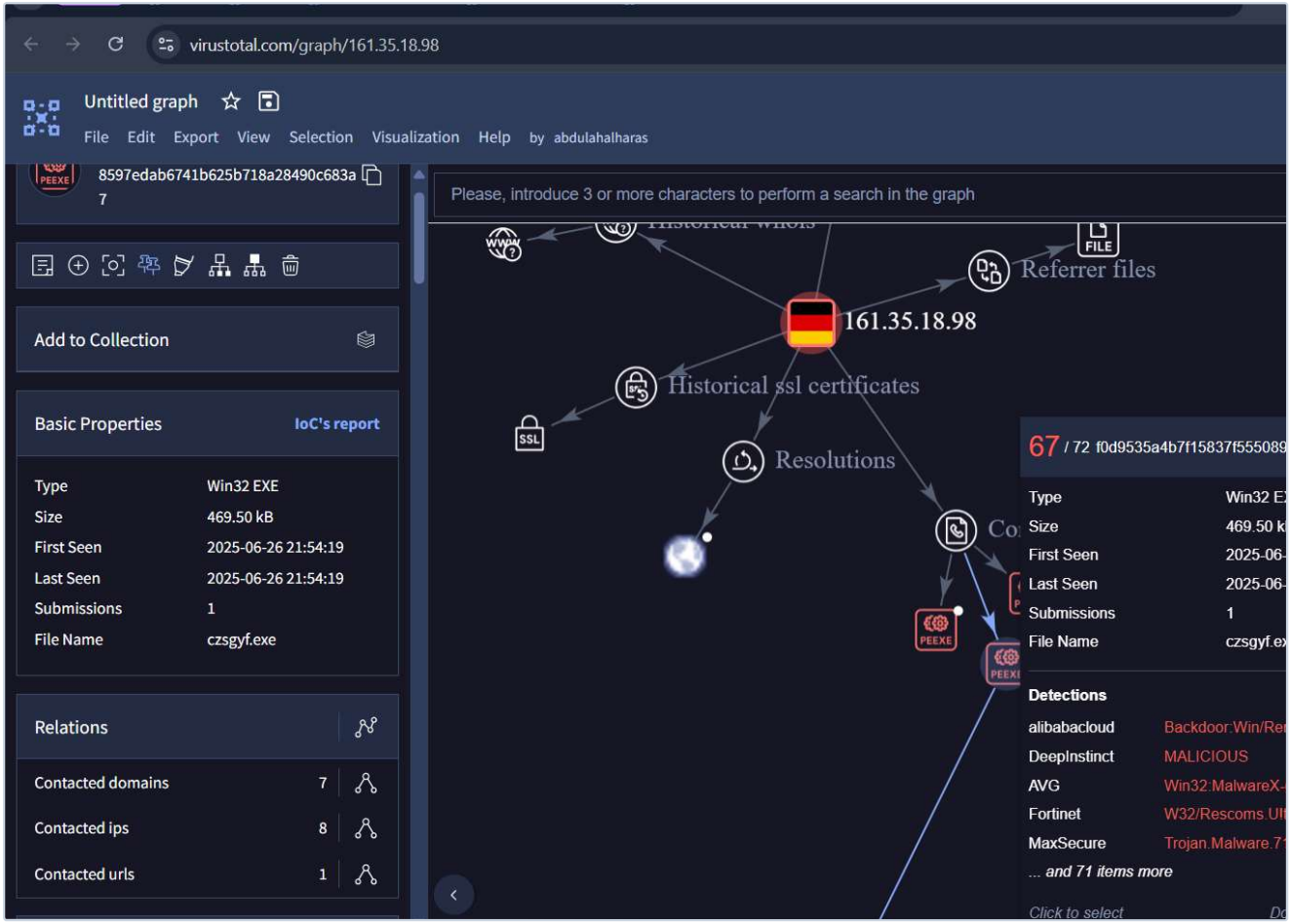


Figure 30. Graph node details for communicating file czsgyf.exe.

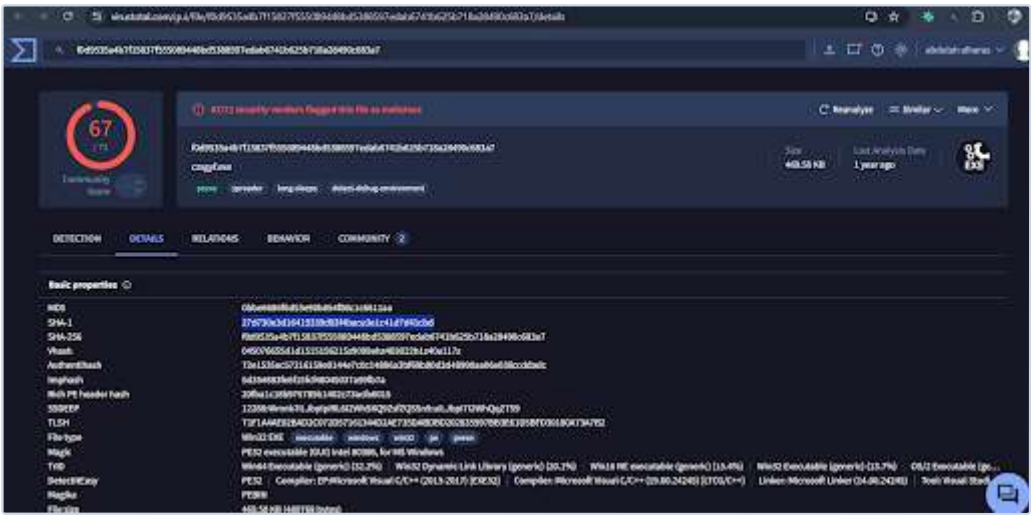


Figure 31. VirusTotal file hash details page for communicating sample czsgyf.exe.

Network analysis of the hollowed `RegAsm.exe` process identified outbound connections to destination IP `161.35.18.98`. VirusTotal correlation identified three communicating malware binaries, including `czsgyf.exe` with SHA-1 hash `27d730e3d16419339d03f4bace3e1c41d7d41cbd`.

5. Attack Timeline

Timestamp (UTC)	Phase	Observed Action
2025-07-11 15:15:28	Initial Access	Malicious tool package downloaded from GitHub repository `mo4de1/Script2Scene`.
2025-07-11 15:16:23	Execution	`Clcw.ps1.ps1` executed via PowerShell with execution policy bypass.
2025-07-11 15:16:35	Payload Staging	Password-protected archive `4359ce22-2f6d-43b8-b34d-063d74715562.7z` dropped in Temp.
2025-07-11 15:18:46	Extraction	`7zInstaller.exe` extracted Electron application contents (`app.asar`).
2025-07-11 15:18:56	Privilege Escalation	`ms-settings` protocol handler hijacked in registry to point to payload.
2025-07-11 15:18:59	Privilege Escalation	`ComputerDefaults.exe` executed, triggering elevated UAC bypass payload.
2025-07-11 15:19:01	Persistence	Three scheduled tasks registered under `Device Guide` key.
2025-07-11 15:19:04	Defense Evasion	Registry keys updated to disable Windows System Restore (`DisableSR = 1`).
2025-07-11 15:20:34	Process Hollowing	Encrypted payloads (`boot_f` , `kernel f` , `thread_f`) decrypted and injected into `RegAsm.exe` .
2025-07-11 15:20:38	Masquerading	`NVIDIA CONTROL PANEL.EXE` executed from user local Vault directory.
2025-07-13 08:40:57	Exfiltration / C2	Outbound HTTPS connection established to `149.154.167.220` (Telegram C2 API).

6. MITRE ATT&CK Mapping

Technique	ID	Observed Execution Context
Supply Chain Compromise	T1195.001	Weaponized open-source GitHub repository used for initial delivery.
Command and Scripting Interpreter	T1059.001	PowerShell scripts (`Clcw.ps1.ps1` , `mbam.ps1`) executed with bypass flags.
Bypass User Account Control	T1548.002	Protocol handler hijacking of `ms-settings` triggered via `ComputerDefaults.exe` .
Scheduled Task	T1053.005	Creation of three persistence tasks under `Microsoft\Windows\Device Guide` .
Inhibit System Recovery	T1490	Modification of `SystemRestore` registry policies to disable recovery points.
Process Injection: Process Hollowing	T1055.012	In-memory decryption and injection of PE payloads into `RegAsm.exe` .
Masquerading	T1036.005	Payload named `NVIDIA CONTROL PANEL.EXE` running outside standard installation paths.
Web Service: Exfiltration to Cloud Storage	T1567.002	Encrypted C2 and data exfiltration conducted over Telegram Bot API (`api.telegram.org`).

7. Indicators of Compromise (IoCs)

Indicator Type	Value	Notes / Context
Repository URL	`https://github.com/mo4de1/Script2Scene`	Initial access source repository
Script Name	`Clcw.ps1.ps1`	First-stage PowerShell dropper
Archive Container	`4359ce22-2f6d-43b8-b34d-063d74715562.7z`	Payload container archive
File MD5	`BD8CB26657654E9CF107AF2218564AB2`	MD5 hash of extracted `main.js`
File SHA-1	`27d730e3d16419339d03f4bace3e1c41d7d41cbd`	SHA-1 hash of communicating binary `czsgyf.exe`
Scheduled Tasks	`Register DeviceSecurityAlert` `Register DevicePowerStateChange` `Register DeviceNetworkChange`	Persistence task entries
Registry Key	`HKCU\Software\Classes\ms-settings\shell\open\command`	UAC bypass hijack location
File Path	`AppData\Local\Microsoft\Vault\UserRoamingTiles\NVIDIAContainer\NVIDIA CONTROL PANEL.EXE`	Masqueraded binary location
IP Address / C2	`149.154.167.220`	Telegram API infrastructure (Port 443)
IP Address / C2	`161.35.18.98`	External C2 node (DigitalOcean)

8. Impact Assessment

The attacker achieved high-integrity code execution on the developer workstation, established persistent scheduled tasks, disabled system recovery features, and successfully hollowed legitimate processes for in-memory payload execution. Development credentials, source code repositories, and local system data must be considered compromised. The system could also serve as a pivot point for broader lateral movement across internal development environments.

9. Recommendations

- Isolate the host `DESKTOP-POKO2F4` from the network immediately and revoke all developer credentials and SSH/API keys accessed on this machine.
- Remove the malicious scheduled tasks (`Register DeviceSecurityAlert`, `Register DevicePowerStateChange`, `Register DeviceNetworkChange`) and clean the `ms-settings` registry hijack keys.
- Re-enable Windows System Restore policies by restoring `DisableSR` and `DisableConfig` registry DWORD values to `0`.
- Block network communication to IP addresses `161.35.18.98` and restrict outbound Telegram API traffic to authorized management accounts.
- Deploy EDR rules to detect protocol handler modifications under `HKCU\Software\Classes\*\shell\open\command` followed by execution of auto-elevating binaries like `ComputerDefaults.exe`.
- Perform full system re-imaging of the affected workstation from verified baseline images prior to returning it to active service.

10. Conclusion

The forensic analysis confirms that host `DESKTOP-POKO2F4` suffered a full system compromise originating from a weaponized GitHub supply chain attack. The multi-stage campaign moved from initial download to high-privilege execution, registry manipulation, persistence installation, process hollowing, and covert C2 communication within minutes. Implementing the containment measures, registry repairs, and detection rules outlined above will remediate the host and guard against similar supply chain threats.